

The route is vulnerable to **SQL injection** because user input is concatenated directly into the SQL query. It also exposes the database result directly and appears to compare plaintext passwords.

A safer Express implementation using parameterized queries would be:

```
app.post('/api/v1/login', async (req, res) => {

  const { email, password } = req.body;

  if (!email || !password) {

    return res.status(400).json({

      error: 'Email and password are required'

    });

  }

  const query = `

    SELECT id, name, email, password_hash

    FROM users

    WHERE email = ?

    LIMIT 1

  `;

  try {

    const [rows] = await db.promise().query(query, [email]);
```

```
if (rows.length === 0) {

  return res.status(401).json({

    error: 'Invalid email or password'

  });

}

const user = rows[0];

// Use bcrypt/argon2 to verify the submitted password.

const passwordValid = await bcrypt.compare(

  password,

  user.password_hash

);

if (!passwordValid) {

  return res.status(401).json({

    error: 'Invalid email or password'

  });

}

// Never return password_hash to the client.
```

```

return res.status(200).json({

  id: user.id,

  name: user.name,

  email: user.email

});

} catch (err) {

  console.error('Login error:', err);

  return res.status(500).json({

    error: 'Internal server error'

  });

}

});

```

Key fixes

1. **Parameterized query**
`db.promise().query(query, [email])`
 prevents the email from being interpreted as SQL syntax.
2. **Password hashing**
 Store a `password_hash` generated with a password-hashing algorithm such as **bcrypt** or **Argon2**. Don't store or query plaintext passwords.
3. **Generic authentication error**
 Return the same `Invalid email or password` message for both nonexistent users and incorrect passwords.
4. **Don't expose database records**
 The vulnerable code does:
`res.send(result);`
 which can expose sensitive fields such as password hashes. Return only the fields the client actually needs.

5. Don't use `throw err` inside the callback

Handle the error and return an appropriate HTTP response instead of potentially crashing the process.

For a production API, the successful login would typically also create a **session or signed JWT**, rather than simply returning user information.

Security Audit

The revised endpoint has addressed the two major vulnerabilities in the original version: **SQL injection** and **unhandled database errors**. However, several security improvements are still appropriate.

Finding	Severity	Risk	Recommendation
SQL injection	Low / Mitigated	User input is passed as a parameter rather than concatenated into SQL.	Keep parameterized queries.
Unhandled database crash	Low / Mitigated	Database errors are caught and converted to HTTP 500 responses.	Keep the <code>try/catch</code> .
Password handling	Low / Conditional	<code>bcrypt.compare()</code> is appropriate, but security depends on passwords having been stored using a strong password-hashing algorithm.	Use bcrypt or Argon2id with appropriate parameters.
User enumeration	Low	The endpoint currently uses the same error for nonexistent users and incorrect passwords, which is good. However, timing differences can still potentially leak information.	Consider rate limiting and, where appropriate, additional timing protections.
Brute-force attacks	Medium	An attacker can repeatedly attempt passwords if no rate limit exists.	Add per-IP/account rate limiting and monitoring.
Sensitive logging	Low	<code>console.error()</code> logs the error object. Depending on the database/driver, errors could contain implementation details.	Use structured logging and avoid exposing sensitive data.

Authentication/session handling	Medium / Conditional	Successful authentication only returns user information; no session or access token is established.	Create a secure session or short-lived access token after authentication.
Input validation	Low	Only presence is checked. Malformed or excessively large input can reach the database/hash comparison.	Validate email format and impose reasonable input-length limits.

Refactored secure version

```
const bcrypt = require('bcrypt');

app.post('/api/v1/login', async (req, res) => {
  try {
    const { email, password } = req.body;

    // Basic input validation
    if (
      typeof email !== 'string' ||
      typeof password !== 'string' ||
      !email.trim() ||
      !password
    ) {
      return res.status(400).json({
        error: 'Email and password are required'
      });
    }

    // Optional defensive limits
    if (email.length > 254 || password.length > 128) {
      return res.status(400).json({
        error: 'Invalid credentials'
      });
    }

    // Parameterized query prevents SQL injection.
    const query = `
      SELECT id, name, email, password_hash
      FROM users
      WHERE email = ?
      LIMIT 1
    `;

    const [rows] = await db.promise().query(query, [email.trim()]);
```

```

// Deliberately use the same response for both cases.
if (rows.length === 0) {
  return res.status(401).json({
    error: 'Invalid email or password'
  });
}

const user = rows[0];

// Compare against the stored password hash.
const passwordValid = await bcrypt.compare(
  password,
  user.password_hash
);

if (!passwordValid) {
  return res.status(401).json({
    error: 'Invalid email or password'
  });
}

// Do not expose password_hash.
return res.status(200).json({
  id: user.id,
  name: user.name,
  email: user.email
});

} catch (err) {
  // Log details internally; don't expose them to the client.
  console.error('Login error:', err);

  return res.status(500).json({
    error: 'Internal server error'
  });
}
});

```

Important remaining production controls

The code is substantially safer, but a production login endpoint should additionally have:

- **Rate limiting** to reduce credential-stuffing/brute-force attacks.
- **HTTPS/TLS** so credentials aren't transmitted in plaintext.
- **Secure session cookies** (**HttpOnly**, **Secure**, appropriate **SameSite**) if using cookie-based sessions.
- **Argon2id or bcrypt** for password storage; never plaintext or reversible encryption.
- **Generic authentication errors**, as implemented above.

- **Audit/security logging** without logging passwords, tokens, or other credentials.
- **CSRF protection** if authentication uses cookies and the API is exposed to browser-based cross-site requests.
- **A session/token issued after successful authentication**; returning the user record alone does not actually establish an authenticated session.

Audit conclusion: The provided version already mitigates the original **SQL injection** and **unhandled database-error** vulnerabilities. The principal remaining concerns are **brute-force protection, secure authentication/session establishment, input validation, and operational logging**.